SKILL

User Input

```
$ARGUMENTS
```

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before specification): - Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_specify` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (`.`) with hyphens (`-`). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): ``` ## Extension Hooks

```
**Optional Pre-Hook**: {extension}
Command: `/{command}`
Description: {description}

Prompt: {prompt}
To execute: `/{command}`
```
```

- **Mandatory hook** (`optional: false`):

  ```
  ## Extension Hooks

  Automatic Pre-Hook: {extension}
  Executing: `/{command}`
  ```
  ```

```
EXECUTE_COMMAND: {command}

Wait for the result of the hook command before proceeding to
the Outline.
```

After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Outline

The text the user typed after `/skill:speckit-specify` in the triggering message **is** the feature description. Assume you always have it available in this conversation even if `$ARGUMENTS` appears literally below. Do not ask the user to repeat it unless they provided an empty command.

Given that feature description, do this:

1. **Generate a concise short name** (2-4 words) for the feature:
    - Analyze the feature description and extract the most meaningful keywords
    - Create a 2-4 word short name that captures the essence of the feature
    - Use action-noun format when possible (e.g., “add-user-auth”, “fix-payment-bug”)
    - Preserve technical terms and acronyms (OAuth2, API, JWT, etc.)
    - Keep it concise but descriptive enough to understand the feature at a glance
    - Examples:
        - “I want to add user authentication” → “user-auth”
        - “Implement OAuth2 integration for the API” → “oauth2-api-integration”
        - “Create a dashboard for analytics” → “analytics-dashboard”
        - “Fix payment processing timeout bug” → “fix-payment-timeout”
2. **Branch creation** (optional, via hook):

If a `before_specify` hook ran successfully in the Pre-Execution Checks above, it will have created/switched to a git branch and output JSON containing `BRANCH_NAME` and `FEATURE_NUM`. Note these values for reference, but the branch name does **not** dictate the spec directory name.

If the user explicitly provided `GIT_BRANCH_NAME`, pass it through to the hook so the branch script uses the exact value as the branch name (bypassing all prefix/suffix generation).

3. **Create the spec feature directory**:

   Specs live under the default `specs/` directory unless the user explicitly provides `SPECIFY_FEATURE_DIRECTORY`.

   Resolution order for `SPECIFY_FEATURE_DIRECTORY`:

   1. If the user explicitly provided `SPECIFY_FEATURE_DIRECTORY` (e.g., via environment variable, argument, or configuration), use it as-is
   2. Otherwise, auto-generate it under `specs/`:
      - Check `.specify/init-options.json` for `feature_numbering` (preferred) or `branch_numbering` (deprecated, migration only — will be removed in a future release)
      - If `"timestamp"`: prefix is `YYYYMMDD-HHMMSS` (current timestamp)
      - If `"sequential"` or absent: prefix is `NNN` (next available 3-digit number after scanning existing directories in `specs/`)
      - Construct the directory name: `<prefix>-<short-name>` (e.g., `003-user-auth` or `20260319-143022-user-auth`)
      - Set `SPECIFY_FEATURE_DIRECTORY` to `specs/<directory-name>`
      - If `branch_numbering` was used (and `feature_numbering` was absent), emit a one-line warning: “ `branch_numbering` in init-options.json is deprecated. Rename to `feature_numbering`.”

   Create the directory and spec file:

   - `mkdir -p SPECIFY_FEATURE_DIRECTORY`
   - Resolve the active `spec-template` through the Spec Kit preset/template resolution stack (equivalent to `specify preset resolve spec-template`)
   - Copy the resolved `spec-template` file to `SPECIFY_FEATURE_DIRECTORY/spec.md` as the starting point
   - Set `SPEC_FILE` to `SPECIFY_FEATURE_DIRECTORY/spec.md`
   - Persist the resolved path to `.specify/feature.json`:

```
{
  "feature_directory": "<resolved feature dir>"
}
```

Write the actual resolved directory path value (for example, `specs/003-user-auth`), not the literal string `SPECIFY_FEATURE_DIRECTORY`. This allows downstream commands (`/skill:speckit-plan`, `/skill:speckit-tasks`, etc.) to locate the feature directory without relying on git branch name conventions.

IMPORTANT:

- You must only create one feature per `/skill:speckit-specify` invocation
- The spec directory name and the git branch name are independent — they may be the same but that is the user's choice
- The spec directory and file are always created by this command, never by the hook

4. Load the resolved active `spec-template` file to understand required sections.

5. **IF EXISTS**: Load `.specify/memory/constitution.md` for project principles and governance constraints.

6. Follow this execution flow:

    1. Parse user description from arguments If empty: ERROR "No feature description provided"
    2. Extract key concepts from description Identify: actors, actions, data, constraints
    3. For unclear aspects:
        - Make informed guesses based on context and industry standards
        - Only mark with [NEEDS CLARIFICATION: specific question] if:
            - The choice significantly impacts feature scope or user experience
            - Multiple reasonable interpretations exist with different implications
            - No reasonable default exists
        - **LIMIT: Maximum 3 [NEEDS CLARIFICATION] markers total**
        - Prioritize clarifications by impact: scope > security/ privacy > user experience > technical details
    4. Fill User Scenarios & Testing section If no clear user flow: ERROR "Cannot determine user scenarios"
    5. Generate Functional Requirements Each requirement must be testable Use reasonable defaults for unspecified details (document assumptions in Assumptions section)

6. Define Success Criteria Create measurable, technology-agnostic outcomes Include both quantitative metrics (time, performance, volume) and qualitative measures (user satisfaction, task completion) Each criterion must be verifiable without implementation details
7. Identify Key Entities (if data involved)
8. Return: SUCCESS (spec ready for planning)

7. Write the specification to SPEC_FILE using the template structure, replacing placeholders with concrete details derived from the feature description (arguments) while preserving section order and headings.

8. **Specification Quality Validation**: After writing the initial spec, validate it against quality criteria:

    1. **Create Spec Quality Checklist**: Generate a checklist file at `SPECIFY_FEATURE_DIRECTORY/checklists/requirements.md` using the checklist template structure with these validation items:

```
# Specification Quality Checklist: [FEATURE NAME]

**Purpose**: Validate specification completeness and
        quality before proceeding to planning
**Created**: [DATE]
**Feature**: [Link to spec.md]

## Content Quality

- [ ] No implementation details (languages, frameworks,
        APIs)
- [ ] Focused on user value and business needs
- [ ] Written for non-technical stakeholders
- [ ] All mandatory sections completed

## Requirement Completeness

- [ ] No [NEEDS CLARIFICATION] markers remain
- [ ] Requirements are testable and unambiguous
- [ ] Success criteria are measurable
- [ ] Success criteria are technology-agnostic (no
        implementation details)
- [ ] All acceptance scenarios are defined
- [ ] Edge cases are identified
- [ ] Scope is clearly bounded
- [ ] Dependencies and assumptions identified

## Feature Readiness

- [ ] All functional requirements have clear acceptance
        criteria
- [ ] User scenarios cover primary flows
```

```
- [ ] Feature meets measurable outcomes defined in Success
      Criteria
- [ ] No implementation details leak into specification

## Notes

- Items marked incomplete require spec updates before `/
      skill:speckit-clarify` or `/skill:speckit-plan`
```

2. **Run Validation Check**: Review the spec against each checklist item:
   - For each item, determine if it passes or fails
   - Document specific issues found (quote relevant spec sections)

3. **Handle Validation Results**:
   - **If all items pass**: Mark checklist complete and proceed to the Mandatory Post-Execution Hooks section
   - **If items fail (excluding [NEEDS CLARIFICATION])**:
     1. List the failing items and specific issues
     2. Update the spec to address each issue
     3. Re-run validation until all items pass (max 3 iterations)
     4. If still failing after 3 iterations, document remaining issues in checklist notes and warn user
   - **If [NEEDS CLARIFICATION] markers remain**:
     1. Extract all [NEEDS CLARIFICATION: ...] markers from the spec
     2. **LIMIT CHECK**: If more than 3 markers exist, keep only the 3 most critical (by scope/security/UX impact) and make informed guesses for the rest
     3. For each clarification needed (max 3), present options to user in this format:

```
## Question [N]: [Topic]

**Context**: [Quote relevant spec section]

**What we need to know**: [Specific question from
        NEEDS CLARIFICATION marker]

**Suggested Answers**:

| Option | Answer | Implications |
|--------|--------|--------------|
```

```
| A      | [First suggested answer] | [What this
       means for the feature] |
| B      | [Second suggested answer] | [What this
       means for the feature] |
| C      | [Third suggested answer] | [What this
       means for the feature] |
| Custom | Provide your own answer | [Explain how to
       provide custom input] |

**Your choice**: _[Wait for user response]_
```

4. **CRITICAL - Table Formatting**: Ensure markdown tables are properly formatted:
   - Use consistent spacing with pipes aligned
   - Each cell should have spaces around content: `| Content |` not `|Content|`
   - Header separator must have at least 3 dashes: `|--------|`
   - Test that the table renders correctly in markdown preview
5. Number questions sequentially (Q1, Q2, Q3 - max 3 total)
6. Present all questions together before waiting for responses
7. Wait for user to respond with their choices for all questions (e.g., “Q1: A, Q2: Custom - [details], Q3: B”)
8. Update the spec by replacing each [NEEDS CLARIFICATION] marker with the user’s selected or provided answer
9. Re-run validation after all clarifications are resolved

4. **Update Checklist**: After each validation iteration, update the checklist file with current pass/fail status

Mandatory Post-Execution Hooks

You MUST complete this section before reporting completion to the user.

Check if `.specify/extensions.yml` exists in the project root. - If it does not exist, or no hooks are registered under `hooks.after_specify`, skip to the Completion Report. - If it exists, read it and look for entries under the `hooks.after_specify` key. - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue to the Completion Report. - Filter out hooks

where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (`.`) with hyphens (`-`). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its `optional` flag: - **Mandatory hook** (`optional: false`) — **You MUST emit `EXECUTE_COMMAND:` for each mandatory hook**: ``` ## Extension Hooks

```
**Automatic Hook**: {extension}
Executing: `/{command}`
EXECUTE_COMMAND: {command}
```
After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- **Optional hook** (`optional: true`):

  ```
  ## Extension Hooks

  **Optional Hook**: {extension}
  Command: `/{command}`
  Description: {description}

  Prompt: {prompt}
  To execute: `/{command}`
  ```

Completion Report

Report completion to the user with: - `SPECIFY_FEATURE_DIRECTORY` — the feature directory path - `SPEC_FILE` — the spec file path - Checklist results summary - Readiness for the next phase (`/skill:speckit-clarify` or `/skill:speckit-plan`)

NOTE: Branch creation is handled by the `before_specify` hook (git extension). Spec directory and file creation are always handled by this core command.

Quick Guidelines

- Focus on **WHAT** users need and **WHY**.
- Avoid HOW to implement (no tech stack, APIs, code structure).
- Written for business stakeholders, not developers.
- DO NOT create any checklists that are embedded in the spec. That will be a separate command.

Section Requirements

- **Mandatory sections**: Must be completed for every feature
- **Optional sections**: Include only when relevant to the feature
- When a section doesn't apply, remove it entirely (don't leave as "N/A")

For AI Generation

When creating this spec from a user prompt:

1. **Make informed guesses**: Use context, industry standards, and common patterns to fill gaps
2. **Document assumptions**: Record reasonable defaults in the Assumptions section
3. **Limit clarifications**: Maximum 3 [NEEDS CLARIFICATION] markers - use only for critical decisions that:
   - Significantly impact feature scope or user experience
   - Have multiple reasonable interpretations with different implications
   - Lack any reasonable default
4. **Prioritize clarifications**: scope > security/privacy > user experience > technical details
5. **Think like a tester**: Every vague requirement should fail the "testable and unambiguous" checklist item
6. **Common areas needing clarification** (only if no reasonable default exists):
   - Feature scope and boundaries (include/exclude specific use cases)
   - User types and permissions (if multiple conflicting interpretations possible)
   - Security/compliance requirements (when legally/financially significant)

Examples of reasonable defaults (don't ask about these):

- Data retention: Industry-standard practices for the domain
- Performance targets: Standard web/mobile app expectations unless specified
- Error handling: User-friendly messages with appropriate fallbacks

- Authentication method: Standard session-based or OAuth2 for web apps
- Integration patterns: Use project-appropriate patterns (REST/ GraphQL for web services, function calls for libraries, CLI args for tools, etc.)

Success Criteria Guidelines

Success criteria must be:

1. **Measurable**: Include specific metrics (time, percentage, count, rate)
2. **Technology-agnostic**: No mention of frameworks, languages, databases, or tools
3. **User-focused**: Describe outcomes from user/business perspective, not system internals
4. **Verifiable**: Can be tested/validated without knowing implementation details

Good examples:

- “Users can complete checkout in under 3 minutes”
- “System supports 10,000 concurrent users”
- “95% of searches return results in under 1 second”
- “Task completion rate improves by 40%”

Bad examples (implementation-focused):

- “API response time is under 200ms” (too technical, use “Users see results instantly”)
- “Database can handle 1000 TPS” (implementation detail, use user-facing metric)
- “React components render efficiently” (framework-specific)
- “Redis cache hit rate above 80%” (technology-specific)

Done When

- [ ] Specification written to `SPEC_FILE` and validated against quality checklist
- [ ] Extension hooks dispatched or skipped according to the rules in Mandatory Post-Execution Hooks above
- [ ] Completion reported to user with feature directory, spec file path, and checklist results